

DataShield — Руководство программиста

Структура решения, логика работы каждого модуля, потоковая конвейерная модель, тесты и сборка. Алгоритмы (накопление, перепривязка, сборка) детально разобраны в `AlgorithmGuide.ru.md`; здесь — архитектура и модули.

1. Обзор решения

`DataShield.slnx` содержит 19 проектов. Конвейер декодирования собран из небольших сборок, каждая решает одну задачу:

Проект	Назначение и логика
<code>DataShield.Interfaces</code>	Контракт конвейера: <code>IDataSource</code> , <code>IDataProcessor</code> , <code>IDataWriter</code> , база <code>DataProcessorBase</code> , делегаты <code>DataReadyHandler</code> , <code>TakeBufferDelegate</code>
<code>DataShield.Codec.Packets</code>	Формат провода: константы <code>PacketFormat</code> (75 Б/100 символов, раскладки H/D) и сериализация заголовка <code>HeaderContent</code>
<code>DataShield.Codec.IO</code>	Источники: <code>FileSource</code> , <code>StreamSource</code> , <code>ByteArraySource</code> (наследники <code>BufferedSourceBase</code>); приёмники: <code>FileDataWriter</code> , <code>StreamDataWriter</code> , <code>ByteListWriter</code> , <code>PreallocatedBufferWriter</code> (наследники <code>WriterBase</code>)
<code>DataShield.Codec.StreamFilter</code>	<code>ByteRangeFilter</code> — фильтр байтов по карте <code>bool[256]</code> из <code>ByteRange</code> ; фабрика <code>CreateBase64()</code>
<code>DataShield.Codec.StreamScanner</code>	<code>SlidingWindowScanner</code> — сканирование скользящим окном с делегатом <code>WindowHandler</code> ; удержание всего потока; очередь отложенных rescan
<code>DataShield.Codec.StreamProcessor</code>	Ядро приёма: накопитель <code>StreamProcessor</code> , слот <code>ReceptionSlot</code> , комбинаторика <code>Versions/*</code> , RS-адаптер <code>RsCodecAdapter</code> , <code>Localization</code> , <code>Progress</code>
<code>DataShield.Codec</code>	Фасад: <code>FileEncoder</code> , <code>FileDecoder</code> , статистика <code>EncodeStats</code> , вывод <code>Packets/PacketIO</code> , <code>OutputFormat(Config)</code>
<code>DataShield.GUI</code>	Avalonia UI (MVVM): <code>MainViewModel</code> , <code>MainWindow.axaml</code> , <code>SectorMapControl</code> , локализация <code>UiStrings</code> / <code>LanguageManager</code> , настройки <code>AppSettings</code>
<code>DataShield.Demo</code>	Непрерывный рандомизированный стенд стабильности/производительности (PASS/WARN/FAIL)
<code>DataShield.Tests</code>	Интеграционные тесты кодека (162)
<code>DataShield.TestsHarness</code>	Источники повреждений для тестов и стенда: <code>DamageEngine</code> , <code>BinaryDamage</code> , <code>PacketDamage</code> , <code>LineDamage</code> , <code>DamageBits</code> , <code>PacketProbe</code> , <code>RandomInput</code>
<code>RsRaid16Demo</code>	Демонстрация и тесты поля GF(2 ¹⁶)/Рида–Соломона
<code>Sha256CompactDemo</code>	Демонстрация компактной реализации SHA-256
<code>DataShield.*.Tests</code> × 6	Юнит-тесты по сборке на каждый модуль конвейера (см. раздел 8)

Каталог `refs-src/` — немодифицируемые справочные исходники `RsRaid16.cs`, `GF16.cs`, `Sha256Compact.cs`; они подключаются компиляцией в проекты, которым нужны RS/SHA. Правки справочников не допускаются.

Граф зависимостей (упрощённо):

```

Interfaces ← Codec.IO ←
Interfaces ← StreamFilter ←
Interfaces ← StreamScanner ← Codec ← GUI, Demo, Tests
Interfaces ← StreamProcessor ← (TestsHarness ← Demo, Tests)
Codec.Packets ←

```

Важно про namespace: типы сборки `DataShield.Codec.StreamProcessor` живут в namespace `DataShield.Codec` / `DataShield.Codec.Versions` — фасадные namespace сохранены при декомпозиции монолита. Пространство имён ≠ имя сборки.

2. Контракт конвейера (`DataShield.Interfaces`)

Всё движение данных построено на трёх ролях:

- **IDataSource** — источник с буферизацией и событийной выдачей. `Start()` запускает чтение в буфер `BufferSize`; заполненный буфер объявляется событием `DataReady`, чтение приостанавливается до вычитки буфера клиентом (естественный backpressure — источник не забегает вперёд). По EOF выбрасывается остаток и источник останавливается сам. `Stop()` — внешняя остановка; остаток буфера всё равно отдаётся.
- **IDataProcessor : IDataSource** — «чёрный ящик»: имеет вход (`Attach/Detach(IDataSource)`) и собственный выход; результат публикуется своим `DataReady`, что позволяет выстраивать цепочки. `Complete()` — сигнал конца входа: остаток выходного буфера выбрасывается, обработчик останавливается.
- **IDataWriter** — тупиковый приёмник: `Write(ReadOnlySpan<byte>)`, подключение к источнику через `Attach`.

`DataProcessorBase` — общая реализация обработчика: подключение к upstream с каскадными `Start/Stop/Complete` (сигнал конца проходит по всей цепочке), выходной буфер `BufferSize` и два способа выдачи:

- `Emit(bytes)` — байтовый поток, буферизация допустима (склеивать/резать порции можно как угодно);
- `EmitPacket(bytes)` — **неделимые** порции (пакеты): порезать нельзя, выдать нужно целиком.

Потокобезопасность обеспечивается `SyncRoot`; конвейер в целом однопоточный по данным (вызовы идут по цепочке событий), поэтому синхронизация нужна только там, где внешний код читает состояние параллельно (например, `StreamProcessor`).

3. `Codec.Packets` — формат провода

`PacketFormat` — только константы (размеры, смещения полей H1–H5/D1–D3, `Base64Size = 100`, `MaxFileSizeField = 16 777 215`). `HeaderContent` — readonly-record-структура с сериализацией в 51 байт (`ToBytes/WriteTo/ReadFrom`, UInt24LE для размера, имя с паддингом

пробелами) и вычисляемыми `DataVolumeCount` / `TotalVolumeCount`. `PacketHasher` вычисляет усечённые SHA-256-хеши целостности ($H_5 = 24$ байта над содержимым заголовка, $D_3 = 9$ байт над $H_5 \parallel D_1 \parallel D_2$). `FileNameCodec` упаковывает имя в 14-байтное поле H_1 . Логика принятия решений здесь нет — модуль не зависит ни от чего, кроме BCL.

4. `Codec.IO` — источники и приёмники

Источники (`BufferedSourceBase`) реализуют событийную модель `IDataSource` поверх конкретного хранилища: массив (`ByteArraySource`), поток (`StreamSource`), файл (`FileSource`). Приёмники (`WriterBase`) пишут в файл/поток/список/предвыделенный буфер. Используются фасадом `FileDecoder` (вход: `ByteArraySource`, `StreamSource`) и потребителями вывода.

5. `Codec.StreamFilter` и `Codec.StreamScanner`

`ByteRangeFilter` — `IDataProcessor` с картой допустимых байтов `bool[256]`, построенной из набора `ByteRange`. Пропускает байты, входящие в диапазоны, остальные отбрасывает. `CreateBase64()` — готовый фильтр алфавита Base64 для txt-режима (переводы строк, пробелы и мусор исчезают до декодирования). В binary-режиме фильтр в цепочку не включается.

`SlidingWindowScanner` — `IDataProcessor`, обрабатывающий вход окном фиксированной длины (100 байт для Base64, 75 для binary) с делегатом `WindowHandler(window, out emitted) → int`:

- возвращённое значение — продвижение потока в байтах, минимум 1; типичный случай: успех → длина пакета, неуспех → 1. Используется только прямым проходом: повторный проход (rescan) его игнорирует и всегда сдвигается на 1 байт;
- `emitted` — копия распознанного пакета (выдаётся через `EmitPacket`) или null.

Ключевая особенность — **удержание всего потока**: сканер не выбрасывает обработанные байты, пока жив. Это даёт возможность `RequestRescan(handler)` — исчерпывающего повторного прохода удержанных данных другим окном: окно проверяется на каждой позиции, продвижение делегата игнорируется (используется для адресной перепривязки секторов, опоздавших от своего заголовка; полнота обязательна, потому что прыжок прямого прохода после успеха может перепрыгнуть начало перекрывающегося валидного пакета). Очередь отложенных rescans привязана к границе последней прямой выдачи: повторный проход не заходит за неё, чтобы не подтверждать одни и те же данные дважды. Событие `ConsumedAdvanced` сообщает прогресс потребления потока.

6. `Codec.StreamProcessor` — ядро приёма

Состав: `StreamProcessor` (накопитель), `ReceptionSlot` (слот файла), `Versions/` (`SectorCombinationMath`, `SectorVersionSearchOptions`, `ChoicePoint`, `SectorVariant`, `SectorVersionInfo`), `RsCodecAdapter` (RS над $GF(2^{16})$, справочники из `refs-src`), `Localization` (`CodecStrings`), `Progress` (`CodecProgress`, `ScaledProgress`, `ProgressThrottle`).

Логика:

- **StreamProcessor : DataProcessorBase** собирает из кусков входа целые 75-байтные пакеты (буфер `_pending` на границах кусков), классифицирует каждый (автономный хеш → заголовок; хеш с сидом H5 → сектор конкретного слота; сектор может подойти несколькими слотам), ведёт список `ReceptionSlot`. Новый заголовок → новый слот + событие `HeaderAccepted(header, headerHash)` (вне блокировки). Свойства-снимки (`Slots`, `FileCount`, счётчики по всем слотам) читаются под блокировкой параллельно с приёмом. `Recognizes(packet)` — предикат распознавания без побочных эффектов для делегата окна сканера.
- **ReceptionSlot** — `SortedDictionary<номер сектора, List<SectorVariant>>`; версии payload отсортированы по убыванию счётчика подтверждений; `AddSector` либо инкрементирует счётчик совпавшей версии (с «всплытием»), либо добавляет новую версию в конец. Даёт метрики (coverage, карты валидности/коллизий) и сборку `TryAssemble` (прямая → RS → перебор/прокрутка комбинаций версий; подробно — `AlgorithmGuide.ru.md`, раздел 9, и монографии `AssemblyGuide.Academic.ru.md` / `AssemblyGuide.Engineer.ru.md`).
- **Versions** — чистая комбинаторика: одометр `AdvanceIndexes`, `CountCombinations` (с насыщением `long.MaxValue`), `CountRotationStates` (НОК циклов с лимитом), `ShouldUseExhaustiveSearch`; лимиты `SectorVersionSearchOptions` (100 000 комбинаций, 100 000 состояний, 30 с).
- **RsCodecAdapter** — кодирование K→M и восстановление стираний для 64-байтных томов (32 независимых GF-символа на том); условие восстановления: стёртых data ≤ доступных ECC; $K+M \leq 65\,535$.

7. Codec (фасад), GUI, Demo

FileEncoder: файл → N data-томов → M ECC-томов → пакеты секторов + N копий заголовка (первый/последний/равномерно). Фазы прогресса 0–10–75–100, вайп промежуточных буферов. `EncodeToText` — Base64-строки; `EncodeWithStats` — + `EncodeStats` (SHA, счётчики, копии заголовков).

FileDecoder: собирает конвейер `источник → (фильтр Base64) → сканер → накопитель`, поддерживает вход строк Base64, сырых байт и `Stream` с явным `OutputFormat`. По `HeaderAccepted` просит оба сканера (txt и bin) адресно перепривязать удержанные данные окном `RebindWindow` (сектор опоздавшего заголовка: диапазон номера + хеш с сидом нового H5). Сборка — `TryAssemble(HeaderContent)`: поиск слота побайтовым сравнением и вызов `ReceptionSlot.TryAssemble`. Состояние приёма доступно через `Slots` / `FileCount` для UI-индикации.

DataShield.GUI — Avalonia-приложение (MVVM без внешних фреймворков): `MainViewModel` + `RelayCommand`, карта секторов `SectorMapControl`, двуязычность `UiStrings` / `LanguageManager` / `UiLanguage`, конвертеры, настройки `AppSettings`, режимы работы `WorkMode`. Все тяжёлые операции кодека — через фасад `DataShield.Codec`.

DataShield.Demo — бесконечный стенд: случайный файл (1 Б–256 КБ, лог-равномерно), ECC 1–200%, случайная маска повреждений из `DamageBits`, матрица ожиданий PASS/WARN/FAIL, кольцевая таблица итераций со скоростями. WARN — намеренный отказ (повреждение сверх

бюджета/подделка), FAIL — дефект кодека. Основной инструмент регрессии; детали легенды — в выводе самого стенда.

8. Тесты

Сборка	Покрытие	Тестов
<code>DataShield.Interfaces.Tests</code>	контракт конвейера, <code>DataProcessorBase</code>	6
<code>DataShield.Codec.Packets.Tests</code>	формат пакета, сериализация заголовка	52
<code>DataShield.Codec.IO.Tests</code>	источники и приёмники	16
<code>DataShield.Codec.StreamFilter.Tests</code>	фильтры диапазонов, Base64-фильтр	8
<code>DataShield.Codec.StreamScanner.Tests</code>	скользящее окно, удержание, исчерпывающий rescan	11
<code>DataShield.Codec.StreamProcessor.Tests</code>	накопитель, слоты, версии, комбинаторика, RS-адаптер, сборка	185
<code>DataShield.Tests</code> (интеграционные)	фасад кодер/декодер, повреждения, многофайловые потоки	162
Всего		440

9. Сборка и запуск

```
dotnet build DataShield.slnx -c Release
dotnet test DataShield.slnx
dotnet run --project DataShield.Demo -c Release
dotnet run --project DataShield.GUI -c Release
```

Требуется .NET 10 SDK. GUI также собирается в self-contained publish по обычным правилам Avalonia.

10. Соглашения

- Комментарии и XML-doc — на русском языке; публикуемые типы документируются полностью.
- Namespace сохраняют исторические фасадные имена (`DataShield.Codec`, `DataShield.Codec.Versions`) независимо от сборки.
- Справочники `refs-src/` не изменяются и не адаптируются.
- Новая логика приёма/сборки покрывается тестами соответствующей `*.Tests`-сборки; изменения фасада — интеграционными тестами `DataShield.Tests`.